

Automation

A - When a new email arrives (V3) - Include Attachments, Only with Attachments, Folder - inbox

B - Initialize variable - uniqueTimestamp, fileNameSaved, RowsProcessed, SuccessCount, ErrorCount, ErrorMessages, createdFileId

C - Mark as read or unread (V3) - @{triggerOutputs()?['body/id']}

D* - Scope - Main Processing

D1 - Apply to each - Attachments

D2 - Condition -

```
and(
  or(
    endsWith(toLower(items('Apply_to_each')?['name']), '.xlsx'),
    endsWith(toLower(items('Apply_to_each')?['name']), '.xls')
  ),
  contains(toLower(items('Apply_to_each')?['name']), 'booking')
)
```

is equal to true.

True -

D3 - Create file

File Name - concat(variables('uniqueTimestamp'), '-', guid(), '-', items('Apply_to_each')?['name']) - **Retry Policy** → **Exponential - 3 - PT10S**

File Content - @{base64ToBinary(items('Apply_to_each')?['contentBytes'])}

D4 - Set variable - fileNameSaved - @{concat(variables('uniqueTimestamp'), items('Apply_to_each')?['name'])}

~~G - Set variable 1 - createdFileId - @{outputs('Create_file')?['body/Id']}~~

D5 - Is XLS File(Condition) - endsWith(toLower(items('Apply_to_each')?['name']), '.xls') is equal to true

if true send an email to sender via Send an email (V2)

if False

D6 - Run script

File - @{outputs('Create_file')?['body/Id']}

Script - mentioned in end

D7 - Compose tableName - @{outputs('Run_script')?['body/result/tableName']}

D8 - List rows present in a table

File - @{outputs('Create_file')}['body/Id']

Table - @{outputs('Compose_tableName')}

D9 - Apply to each 1 - @{outputs('List_rows_present_in_a_table')}['body/value']

D10 - Get file metadata using path 1 -

File Path - /Shared Documents/Company Reports/Master_Current.xlsx

D11 - actualISO -

```
if(
    equals(trim(string(item())?['ActualDeliveryDate']), ''),
    null,
    if(
        contains(string(item())?['ActualDeliveryDate']), '/'),
        formatDateTime(
            concat(
                substring(string(item())?['ActualDeliveryDate']), 6, 4), '-',
                substring(string(item())?['ActualDeliveryDate']), 3, 2), '-',
                substring(string(item())?['ActualDeliveryDate']), 0, 2),
                'T00:00:00Z'
            ),
        'o'
    ),
    if(
        and(
            isnumber(item())?['ActualDeliveryDate'],
            greater(int(item())?['ActualDeliveryDate'], 0)
        ),
        formatDateTime(
            addDays('1899-12-30', int(item())?['ActualDeliveryDate']),
            'o'
        ),
        formatDateTime(string(item())?['ActualDeliveryDate'], 'o')
    )
)
```

D12 - safeDateForMetrics -

```

coalesce(
  if(
    equals(trim(replace(replace(string(item()?['PickupDate']),decodeUriComponent
('%0D'),''),decodeUriComponent('%0A'),'')), ''),
    null,
    if(
      or(contains(string(item()?['PickupDate']),'T'), contains(string(item()?['PickupDate']),'-')),
      formatDateTime(item()?['PickupDate'],'o'),
      if(
        contains(string(item()?['PickupDate']),'/'),
        formatDateTime(
          concat(
            substring(string(item()?['PickupDate']),6,4),'-',
            substring(string(item()?['PickupDate']),3,2),'-',
            substring(string(item()?['PickupDate']),0,2),
            'T00:00:00Z'
          ),
          'o'
        ),
        formatDateTime(addDays('1899-12-30', int(float(string(item()?['PickupDate'])))), 'o')
      )
    )
  ),

  if(
    equals(trim(replace(replace(string(item()?['ExpectedDeliveryDate']),decodeUriComponent('%0D'),''),decodeUriComponent('%0A'),'')), ''),
    null,
    if(
      or(contains(string(item()?['ExpectedDeliveryDate']),'T'), contains(string(item()?['ExpectedDeliveryDate']),'-')),
      formatDateTime(item()?['ExpectedDeliveryDate'],'o'),
      if(
        contains(string(item()?['ExpectedDeliveryDate']),'/'),
        formatDateTime(
          concat(

```

```

        substring(string(item()?['ExpectedDeliveryDate']),6,4),'-',
        substring(string(item()?['ExpectedDeliveryDate']),3,2),'-',
        substring(string(item()?['ExpectedDeliveryDate']),0,2),
        'T00:00:00Z'
    ),
    'o'
),
    formatDateTime(addDays('1899-12-30', int(float(string(item()?['ExpectedDeliveryDate'])))), 'o')
)
)
),
    formatDateTime(triggerOutputs()?['body/receivedDateTime'], 'o')
)

```

D13 - isoThursdayThisWeek -

```

startOfDay(
    addDays(
        if(
            equals(trim(string(item()?['PickupDate'])), ''),
            if(
                equals(trim(string(item()?['ExpectedDeliveryDate'])), ''),
                formatDateTime(triggerOutputs()?['body/receivedDateTime'], 'o'),
                string(item()?['ExpectedDeliveryDate'])
            ),
            string(item()?['PickupDate'])
        ),
        sub(
            3,
            mod(
                add(
                    dayOfWeek(
                        if(
                            equals(trim(string(item()?['PickupDate'])), ''),
                            if(
                                equals(trim(string(item()?['ExpectedDeliveryDate'])), ''),

```

D14 - isoThursdayWeek1 -

Automation

```
)  
)
```

D15 - cmpWeekYear -

```
json(concat(  
  '{"WeekNum": ',  
  string(  
    add(  
      1,  
      int(  
        div(  
          sub(  
            ticks(outputs('isoThursdayThisWeek')),  
            ticks(outputs('isoThursdayWeek1'))  
          ),  
          6048000000000  
        )  
      )  
    ),  
  ),  
  ', "YearNum": ',  
  string(int(formatDateTime(outputs('isoThursdayThisWeek'),'yyyy'))),  
  '}'  
))
```

D16 - cmpTravel

```
json(concat(  
  '{',  
  '"ExpectedTravelDays": ',  
  if(  
    or(  
      equals(trim(string(item()?['ExpectedDeliveryDate'])), ''),  
      empty(triggerOutputs()?['body/receivedDateTime'])  
    ),  
  'null',  
  string(  

```

```

div(
  sub(
    ticks(string(item()?['ExpectedDeliveryDate'])),
    ticks(formatDateTime(triggerOutputs()?['body/receivedDateTime'],'o'))
  ),
  864000000000
)
)
),
', "ActualTravelDays": ',
if(
  or(
    equals(trim(string(item()?['ActualDeliveryDate'])), ''),
    empty(triggerOutputs()?['body/receivedDateTime'])
  ),
  'null',
  string(
    div(
      sub(
        ticks(string(item()?['ActualDeliveryDate'])),
        ticks(formatDateTime(triggerOutputs()?['body/receivedDateTime'],'o'))
      ),
      864000000000
    )
  )
),
', "DaysLate": ',
if(
  or(
    equals(trim(string(item()?['ActualDeliveryDate'])), ''),
    equals(trim(string(item()?['ExpectedDeliveryDate'])), '')
  ),
  'null',
  string(
    max(
      createArray(
        div(
          sub(

```

```

        ticks(string(item()?['ActualDeliveryDate'])),
        ticks(string(item()?['ExpectedDeliveryDate']))
    ),
    8640000000000
),
0
)
)
)
),
}'
))

```

D17 - cmpDIFOT

```

"DIFOT": "@{if(
    or(equals(string(item()?['ExpectedDeliveryDate']), ''), equals(string(outputs('actual
SO')), '')),
    ',
    if(
        lessOrEquals(
            formatDateTime(string(outputs('actualSO')), 'yyyy-MM-dd'),
            formatDateTime(string(item()?['ExpectedDeliveryDate']), 'yyyy-MM-dd')
        ),
        'HIT',
        'MISS'
    )
})}"

```

D18 - cmpProductType - @toLower(coalesce(item()?['ProductType'],'other'))

D19 - recordKey

```

if(
    not(equals(trim(string(item()?['RefNumber'])), '')),
    concat('ref:', toLower(trim(string(item()?['RefNumber'])))),
    if(
        not(equals(trim(string(item()?['SenderRefNumber'])), '')),
        concat('sref:', toLower(trim(string(item()?['SenderRefNumber'])))),

```


~~Table - tblMaster~~

~~Key Column - RecordKey~~

~~Key Value - @{outputs('rkNormalized')}~~

D22 - diag tableRecordKey -

```
@{if(greater(length(body('List_rows_present_in_a_table_1')?['value']),0),
first(body('List_rows_present_in_a_table_1')?['value'])?['RecordKey'], 'NO MATCH')}
```

D23 - Condition 1 - greater(length(body('List_rows_present_in_a_table_1')?['value']), 0) is equal to true

D23a - Update a row - if true

File - /Shared Documents/Company Reports/Master_Current.xlsx

Table - tblMaster

Key Column - RecordKey

Key Value - @{outputs('rkNormalized')}

Mapping parameters as below.

D23b - Add a row into a table - if False

File - /Shared Documents/Company Reports/Master_Current.xlsx

Table - tblMaster

~~~~~

exiting D23 - Condition 1 loop

D24 - cmpSenderEmail - if Z1 and Z2 either gets succeed or skipped

```
@{coalesce(
triggerOutputs()?['body/from/emailAddress/address'],
triggerOutputs()?['body/sender/emailAddress/address'],
string(triggerOutputs()?['body/from']),
string(triggerOutputs()?['body/sender']))
}
}
```

D25- cmpEmailName -

```
@{toLower(
if(
contains(outputs('cmpSenderEmail'),'@'),
concat(split(outputs('cmpSenderEmail'),'@')[0], '_', split(outputs('cmpSenderEmail'),'@')[1]),
outputs('cmpSenderEmail'))
}
```

```

)
}
D26- cmpCustomerNameRaw -
@{coalesce(
if(equals(trim(string(item()?['Sender']))),''), null, trim(string(item()?['Sender']))),
if(equals(trim(string(item()?['SenderName']))),''), null, trim(string(item()?
['SenderName']))),
outputs('cmpEmailName')
)
}

```

D27 - cmpCustomerKey -

```

@{toLower(
trim(
replace(
replace(
replace(
string(outputs('cmpCustomerNameRaw')),
' ', '-'),
':', '-'),
'/', '-'),
'\\', '-')
)
)
}

```

D28 - cmpCustomerFilePath -

```

@{concat('/Customers/', outputs('cmpCustomerKey'), '/', outputs('cmpCustomerKey'),
'.xlsx')}

```

D29 - Entering scope loop

D30 - cmpKeyEcho - @{outputs('cmpCustomerKey')}

D31 - cmpFullPathEcho - @{concat('/Shared Documents/Customers/',  
outputs('cmpCustomerKey'), '/', outputs('cmpCustomerKey'), '.xlsx')}

D32 - Get file content using path

File Path - @{concat('/Shared Documents/Customers/', outputs('cmpCustomerKey'), '/',  
outputs('cmpCustomerKey'), '.xlsx')}

If D32 succeed -

D32a1 - Get file metadata using path

File Path - @{concat('/Shared Documents/Customers/', outputs('cmpCustomerKey'), '/',

```

outputs('cmpCustomerKey'), '.xlsx'))
D32a2 - Set variable 2 - custWorkbookIdentifier -
@{body('Get_file_metadata_using_path')?['Identifier']}
if AL fail -
D32b1 - Create new folder
Folder Path - @{concat('Customers/', outputs('cmpCustomerKey'))}
D32b2 - Get file content using path 1
File Path - /Shared Documents/Templates/CustomerTemplate.xlsx
D32b3 - Get folder metadata using path
Folder Path - /Shared Documents/Customers/@{outputs('cmpCustomerKey')}
D32b4 - 5 sec Delay
D32b5 - Create file 1
Folder Path - /Shared Documents/Customers/@{outputs('cmpCustomerKey')}
File Name - @{concat(outputs('cmpCustomerKey'), '.xlsx')}
File Content - @{body('Get_file_content_using_path_1')}
D32b6 - Get file metadata using path 2
File Path - @{concat('/Shared Documents/Customers/', outputs('cmpCustomerKey'), '/',
outputs('cmpCustomerKey'), '.xlsx')}
D32b7 - Set variable 3 - custWorkbookIdentifier -
@{coalesce(
body('Get_file_metadata_using_path_2')?['Id'],
body('Get_file_metadata_using_path_2')?['ItemId']
)}

D33 - FinalMeta
File Path - @{concat('/Shared Documents/Customers/', outputs('cmpCustomerKey'), '/',
outputs('cmpCustomerKey'), '.xlsx')}
D34 - 5 sec Delay
D35 - Get tables
File - @{body('FinalMeta')?['Path']}
D36 - Add a row into a table 1
File - @{body('FinalMeta')?['Path']}
Table - tblOrders
Row -
{
"DateReceived": "@{formatDateTime(triggerOutputs()?['body/receivedDateTime'],'yyyy-
MM-dd')}",

```

```

"SenderRefNumber": "@{string(item()?['SenderRefNumber'])}",
"RefNumber": "@{string(item()?['RefNumber'])}",
"SenderName": "@{coalesce(if(equals(trim(string(item()?['SenderName'])), ''), null,
trim(string(item()?['SenderName']))), if(equals(trim(coalesce(triggerOutputs()?
['body/from/emailAddress/name'], '')), ''),
replace(replace(split(coalesce(triggerOutputs()?['body/from/emailAddress/address'],
''), '@')[0], ':', ' '), '-', ' '), trim(coalesce(triggerOutputs()?['body/from/emailAddress/name'],
''))))}",
"ReceiverName": "@{string(item()?['ReceiverName'])}",
"ReceiverSuburb": "@{string(item()?['ReceiverSuburb'])}",
"ReceiverState": "@{string(item()?['ReceiverState'])}",
"CtnQty": "@{if(equals(string(item()?['CtnQty']), ''), '', int(float(string(item()?
['CtnQty']))))}",
"PalletQty": "@{if(equals(string(item()?['PalletQty']), ''), '', int(float(string(item()?
['PalletQty']))))}",
"Weight": "@{if(equals(string(item()?['Weight']), ''), '', float(string(item()?['Weight'])))}",
"PickupDate": "@{if(equals(string(item()?['PickupDate']), ''), '',
formatDateTime(string(item()?['PickupDate']), 'yyyy-MM-dd'))}",
"ExpectedDeliveryDate": "@{if(equals(string(item()?['ExpectedDeliveryDate']), ''), '',
formatDateTime(string(item()?['ExpectedDeliveryDate']), 'yyyy-MM-dd'))}",
"ActualDeliveryDate": "@{string(outputs('actualISO'))}",
"DaysLate": "@{string(coalesce(json(outputs('cmpTravel'))?['DaysLate'], ''))}",
"ExpectedTravelDays": "@{string(coalesce(json(outputs('cmpTravel'))?
['ExpectedTravelDays'], ''))}",
"ActualTravelDays": "@{string(coalesce(json(outputs('cmpTravel'))?['ActualTravelDays'],
''))}",
"Notes": "@{string(item()?['Notes'])}",
"ProductType": "@{string(outputs('cmpProductType'))}",
"RecordKey": "@{string(outputs('rkNormalized'))}",
"WeekNum": "@{coalesce(json(outputs('cmpWeekYear'))?['WeekNum'], '')}",
"YearNum": "@{coalesce(json(outputs('cmpWeekYear'))?['YearNum'], '')}",
"DIFOT": "@{string(item()?['DIFOT'])}"
}

```

---

Mapping parameters for Mater sheet as below.  
Parameters mapped.

```

SenderRefNumber - @{{string(item()?['SenderRefNumber'])}}
RefNumber - @{{string(item()?['RefNumber'])}}
SenderEmail - @{{coalesce(
triggerOutputs()?['body/from/emailAddress/address'],
triggerOutputs()?['body/sender/emailAddress/address'],
triggerOutputs()?['body/from'],
triggerOutputs()?['body/sender'],
''
)}}
ReceivedAt - @{{formatDateTime(triggerOutputs()?['body/receivedDateTime'],'o')}}
SenderName - @{{coalesce(
if(equals(trim(string(item()?['SenderName'])), ''), null, trim(string(item()?
['SenderName']))),
if(equals(trim(coalesce(triggerOutputs()?['body/from/emailAddress/name'], '')), ''),
replace(replace(split(coalesce(triggerOutputs()?['body/from/emailAddress/address'],
''), '@')[0], ',', '_'), '-', '' ),
trim(coalesce(triggerOutputs()?['body/from/emailAddress/name'], ''))
)
)}}
ReceiverName - @{{string(item()?['ReceiverName'])}}
ReceiverSuburb - @{{string(item()?['ReceiverSuburb'])}}
ReceiverState - @{{string(item()?['ReceiverState'])}}
CtnQty - @{{if(equals(string(item()?['CtnQty']), ''), '', int(float(string(item()?['CtnQty']))))}}
PalletQty - @{{if(equals(string(item()?['PalletQty']), ''), '', int(float(string(item()?
['PalletQty']))))}}
PalletType - @{{string(item()?['PalletType'])}}
TransferType - @{{string(item()?['TransferType'])}}
Weight - @{{if(equals(string(item()?['Weight']), ''), '', float(string(item()?['Weight'])))}}
PickupDate - @{{formatDateTime(triggerOutputs()?['body/receivedDateTime'], 'yyyy-MM-
dd')}}
PalletDimensions - @{{string(item()?['PalletDimensions'])}}
ExpectedDeliveryDate - @{{if(equals(string(item()?
['ExpectedDeliveryDate']), ''), '', formatDateTime(string(item()?

```

```

['ExpectedDeliveryDate']], 'yyyy-MM-dd'))}
ActualDeliveryDate - @{{outputs('actualISO')}}
Notes - @{{string(item()['Notes'])}}
WeekNum - @{{coalesce(json(outputs('cmpWeekYear'))['WeekNum'], '')}}
YearNum - @{{coalesce(json(outputs('cmpWeekYear'))['YearNum'], '')}}
DaysLate - @{{coalesce(json(outputs('cmpTravel'))['DaysLate'], '')}}
DIFOT - @{{outputs('cmpDIFOT')}}
ProductType - @{{outputs('cmpProductType')}}
ConsignmentNumber - @{{string(item()['ConsignmentNumber'])}}
RecordKey - @{{outputs('rkNormalized')}}

```

## Script

```

/*****
General Transport Booking Form Normalizer
- Works with various booking form formats
- Creates a normalized sheet "_Normalized"
- Creates a table "Data" with canonical headers
- Filters out placeholder/filler rows (e.g., PLEASE SELECT / AUTOFILL with no refs)
- De-duplicates repeated records
- Returns { ok, info, rowsFound, tableName, sheetName }
*****/

function main(workbook: ExcelScript.Workbook): {
    ok: boolean;
    info: string;
    rowsFound: number;
    tableName: string;
    sheetName: string;
} {
    // ===== Configuration =====
    const NORMALIZED_SHEET: string = "_Normalized";
    const TABLE_NAME: string = "Data";

    // Canonical headers (removed WEEKNUM, YEARNUM, DIFOT)

```

```

const TARGET_HEADERS: string[] = [
    "SenderRefNumber", "RefNumber", "SenderEmail", "ReceivedAt", "SourceFileU
RL", "SenderName", "SenderDetails",
    "PickupTimes", "ReceiverName", "ReceiverSuburb", "ReceiverState", "CtnQty",
    "PalletQty",
    "PalletType", "TransferType", "Weight", "PickupDate", "PalletDimensions",
    "ExpectedDeliveryDate", "ActualDeliveryDate", "Notes", "ProductType", "Special
Instructions",
    "DaysLate", "ExpectedTravelDays", "ActualTravelDays"
];

// ===== Helpers =====
function safeString(v: string | number | boolean | null | undefined): string {
    return (v === undefined || v === null) ? "" : String(v).trim();
}

function colLetter(n: number): string {
    let out = "";
    let num = n;
    while (num > 0) {
        const r = (num - 1) % 26;
        out = String.fromCharCode(65 + r) + out;
        num = Math.floor((num - 1) / 26);
    }
    return out;
}

function toISODate(dateStr: string): string {
    if (!dateStr) return "";
    const input = String(dateStr).trim();

    // --- Excel serial dates (guard against blanks/zeros/ancient serials) ---
    // Many forms have an empty date cell formatted as a date → value "0".
    // Excel serials: 25569 = 1970-01-01, 36526 = 2000-01-01.
    // Treat anything before 2000-01-01 or way in the future as empty.
    if (/^\d+$/.test(input)) {
        const serial = parseInt(input, 10);
        if (serial <= 0) return ""; // blank cells often show up as 0
    }
}

```



```

64) if (serial < 36526 || serial > 60000) return ""; // outside sane range (2000..20

const baseUtc = Date.UTC(1899, 11, 30); // Excel's epoch with 1900-leap-bug
compat
const ms = serial * 24 * 60 * 60 * 1000;
const d = new Date(baseUtc + ms);
const year = d.getUTCFullYear();
if (year < 2000 || year > 2100) return "";
return d.toISOString();
}

// --- Explicit formats ---
const patterns: RegExp[] = [
  /^(\d{1,2})([\\\.])(\d{1,2})([\\\.])(\d{4})$/, // dd/mm/yyyy or dd.mm/yyyy
  /^(\d{4})[-\\](\d{1,2})[-\\](\d{1,2})$/ // yyyy-mm-dd or yyyy/mm/dd
];

for (let i = 0; i < patterns.length; i++) {
  const m = input.match(patterns[i]);
  if (m) {
    let day: number, month: number, year: number;
    if (i === 0) {
      day = parseInt(m[1], 10);
      month = parseInt(m[2], 10);
      year = parseInt(m[3], 10);
    } else {
      year = parseInt(m[1], 10);
      month = parseInt(m[2], 10);
      day = parseInt(m[3], 10);
    }
    if (month >= 1 && month <= 12 && day >= 1 && day <= 31 && year >= 2000
&& year <= 2100) {
      const d = new Date(Date.UTC(year, month - 1, day));
      if (!isNaN(d.getTime())) return d.toISOString();
    }
    return "";
  }
}

```

```

    }

    // --- General parse fallback (with year sanity check) ---
    const parsed = new Date(input);
    if (!isNaN(parsed.getTime())) {
        const year = parsed.getUTCFullYear();
        if (year >= 2000 && year <= 2100) return parsed.toISOString();
    }
    return "";
}

function extractISODateFromText(text: string): string {
    if (!text) return "";

    const datePatterns: RegExp[] = [
        /\b(\d{1,2}[\/\.\-]\d{1,2}[\/\.\-]\d{4})\b/g,
        /\b(\d{4}[-\/]\d{1,2}[-\/]\d{1,2})\b/g
    ];

    for (const pattern of datePatterns) {
        const match = text.match(pattern);
        if (match) {
            return toISODate(match[0]);
        }
    }
    return "";
}

function isDataRow(row: (string | number | boolean)[]): boolean {
    // Check if this row contains actual data (not headers, instructions, etc.)
    for (let i = 0; i < row.length; i++) {
        const cell = safeString(row[i]);
        if (cell) {
            const lower = cell.toLowerCase();

            // Skip obvious non-data templates/labels
            if (
                lower.includes("please select") ||

```

```

        lower.includes("sender details") ||
        lower.includes("receiver details") ||
        lower.includes("transport booking") ||
        lower.includes("special instructions") ||
        lower.includes("attn:") ||
        lower.includes("phone:") ||
        lower.includes("email") ||
        lower.includes("pickup date:") ||
        lower.includes("for temp controlled") ||
        lower.includes("for dry freight") ||
        lower === "autofill" ||
        lower === "autofill"
    ) {
        return false;
    }

    // Likely data if it has meaningful content
    if (
        cell.length > 2 &&
        !cell.startsWith("(") &&
        !cell.endsWith(")") &&
        cell.toUpperCase() !== "PLEASE SELECT"
    ) {
        return true;
    }
}
return false;
}

```

```

function detectColumnMapping(headers: string[]): { [key: string]: number } {
    const mapping: { [key: string]: number } = {};
    for (let i = 0; i < headers.length; i++) {
        const header = headers[i].toLowerCase();

        if (header.includes("sender") && header.includes("name")) mapping["sender
Name"] = i;
        else if (header.includes("sender") && header.includes("address")) mapping

```

```

["senderAddress"] = i;
    else if (header.includes("pickup") && header.includes("time")) mapping["pick
upTimes"] = i;
    else if (header.includes("receiver") && header.includes("name")) mapping["re
ceiverName"] = i;
    else if (header.includes("receiver") && header.includes("address")) mapping
["receiverAddress"] = i;
    else if (header.includes("delivery") && header.includes("date")) mapping["del
iveryDate"] = i;
    else if (header.includes("product") && header.includes("type")) mapping["pro
ductType"] = i;
    else if (header.includes("pallet") && header.includes("type")) mapping["pallet
Type"] = i;
    else if (header.includes("transfer") && header.includes("type")) mapping["tra
nsferType"] = i;
    else if (header.includes("pallet") && (header.includes("qty") || header.include
s("quantity") || header.includes("no. of pallets"))) mapping["palletQty"] = i;
    else if (header.includes("weight")) mapping["weight"] = i;
    else if (header.includes("dimension")) mapping["palletDimensions"] = i;
    else if (header.includes("reference") || header.includes("ref")) {
        if (header.includes("sender")) mapping["senderRef"] = i;
        else if (header.includes("receiver")) mapping["receiverRef"] = i;
        else mapping["senderRef"] = i; // Default to sender ref
    }
    else if (header.includes("special") || header.includes("instruction")) mapping
["specialInstructions"] = i;
}
return mapping;
}

```

```

function parseAddress(address: string): { name: string; details: string; suburb: stri
ng; state: string } {
    if (!address) return { name: "", details: "", suburb: "", state: "" };

    const lines = address.split('\n').filter(line => line.trim() !== '');
    if (lines.length === 0) return { name: "", details: "", suburb: "", state: "" };

    let name = "";

```

```

let details = "";
let suburb = "";
let state = "";

// Check if first line contains numbers (likely address, not just name)
const firstLine = lines[0].trim();
const hasNumbersInFirstLine = /\d/.test(firstLine);
const hasUnitInFirstLine = /\b(UNIT|SHOP|LEVEL|FLOOR)\s*\d+/i.test(firstLine);

// NEW: Check if it's a business name with colon (like "Dunya Foods:1/1164")
const hasBusinessNameWithColon = /^[A-Za-z\s]+:\s*\d+/i.test(firstLine);

if ((hasNumbersInFirstLine || hasUnitInFirstLine) && !hasBusinessNameWithColon) {
    // First line contains address elements (but not business name with colon) - treat all lines as address
    name = ""; // No separate name, it's all address
    details = lines.join(', ').trim();
} else {
    // First line is likely just the business name or business name with address
    if (hasBusinessNameWithColon) {
        // Split business name from address at the colon
        const colonIndex = firstLine.indexOf(':');
        name = firstLine.substring(0, colonIndex).trim();
        details = firstLine.substring(colonIndex + 1).trim();

        // Add remaining lines to details
        if (lines.length > 1) {
            details += ', ' + lines.slice(1).join(', ').trim();
        }
    } else {
        // Regular business name without colon
        name = firstLine;
        details = lines.slice(1).join(', ').trim();
    }
}

// Extract state

```

```

    const stateMatch = details.match(/\\b(NSW|VIC|QLD|SA|WA|TAS|ACT|NT|Western Australia|South Australia|Northern Territory)\\b/i);
    if (stateMatch) {
        state = stateMatch[0];
        // Standardize state names
        if (state.toUpperCase() === "VIC") state = "VIC";
        else if (state.toUpperCase() === "NSW") state = "NSW";
        else if (state.toUpperCase() === "QLD") state = "QLD";
        else if (state.toUpperCase() === "SA") state = "SA";
        else if (state.toUpperCase() === "WA") state = "WA";
        else if (state.toUpperCase() === "TAS") state = "TAS";
        else if (state.toUpperCase() === "ACT") state = "ACT";
        else if (state.toUpperCase() === "NT") state = "NT";
        else if (state.toLowerCase() === "western australia") state = "WA";
        else if (state.toLowerCase() === "south australia") state = "SA";
        else if (state.toLowerCase() === "northern territory") state = "NT";
    }

    return { name, details, suburb, state };
}

// ---- Placeholder-aware helpers ----
function isPlaceholderCell(s: string): boolean {
    const up = s.trim().toUpperCase();
    return (
        up === "PLEASE SELECT" ||
        up === "AUTOFILL" ||
        up === "'AUTOFILL'" ||
        up === "'PLEASE SELECT'"
    );
}

function isReferenceLike(s: string): boolean {
    const up = s.toUpperCase();
    return (
        /\\bSO\\d{5,}\\b/.test(up) || // SO132982
        /\\bNC\\d{5,}\\b/.test(up) || // NC36016858
        /\\b\\d{6,}\\b/.test(up) // long numeric POs, etc.
    );
}

```

```

    );
}

function hasReceiverInfo(outRow: string[], H: { [k: string]: number }): boolean {
    return !(outRow[H["ReceiverName"]] || outRow[H["ReceiverSuburb"]] || outRow[H["ReceiverState"]]);
}

function hasQuantities(outRow: string[], H: { [k: string]: number }): boolean {
    return !(outRow[H["PalletQty"]] || outRow[H["Weight"]] || outRow[H["PalletDimensions"]]);
}

function hasAnyRef(outRow: string[], H: { [k: string]: number }): boolean {
    return !(outRow[H["SenderRefNumber"]] || outRow[H["RefNumber"]]);
}

/** Final gatekeeper for normalized rows */
function isValidNormalizedRow(outRow: string[], H: { [k: string]: number }): boolean {
    // Must have some receiver info and either a reference or quantities.
    if (!hasReceiverInfo(outRow, H)) return false;
    if (!(hasAnyRef(outRow, H) || hasQuantities(outRow, H))) return false;

    // Allow placeholders in descriptive fields as long as refs/qty + receiver exist
    return true;
}

/** Detect obviously filler input rows before mapping */
function looksLikeFillerInputRow(row: (string | number | boolean)[]): boolean {
    const upCells = row.map(c => safeString(c).toUpperCase());
    const hasPlaceholder = upCells.some(c => c.includes("PLEASE SELECT")) || c.includes("AUTOFILL");
    const hasRefToken = upCells.some(c => isReferenceLike(c));
    // If it has placeholders but no reference tokens, treat as filler row
    return hasPlaceholder && !hasRefToken;
}

```

```

// ===== Main Execution =====

// Select source sheet
const allSheets = workbook.getWorksheets();
let sourceSheet: ExcelScript.Worksheet | null = null;

for (let i = 0; i < allSheets.length; i++) {
    const sheet = allSheets[i];
    const name = sheet.getName();
    if (name !== NORMALIZED_SHEET && sheet.getVisibility() !== ExcelScript.Sheet
Visibility.hidden) {
        sourceSheet = sheet;
        break;
    }
}

if (!sourceSheet) {
    return { ok: false, info: "No suitable worksheet found", rowsFound: 0, tableNam
e: "", sheetName: "" };
}

const srcName = sourceSheet.getName();
console.log("Processing sheet: " + srcName);

// Read data
const dataRows: string[][] = [];
const usedRange = sourceSheet.getUsedRange();

if (!usedRange) {
    return { ok: false, info: "No data found in sheet", rowsFound: 0, tableName: "", s
heetName: "" };
}

const usedVals = usedRange.getValues() as (string | number | boolean)[][];
const totalRows = usedVals.length;
const totalCols = usedVals[0]?.length || 0;

console.log(`Sheet dimensions: ${totalRows} rows x ${totalCols} columns`);

```



```

// ===== NEW: Detect a global Pickup Date from the top-right (below phone/email) =====
// We scan a compact "header" region in the top-right corner for any date text.
// This serves as a fallback for rows that don't contain their own pickup date.
let globalPickupDate = "";
try {
    const headerRowsToScan = Math.min(12, totalRows); // a bit generous
    const headerColsToScan = Math.min(6, Math.max(1, totalCols)); // up to 6 right
most columns
    const startCol = Math.max(0, totalCols - headerColsToScan);

    const headerRange = sourceSheet.getRangeByIndexes(0, startCol, headerRows
ToScan, headerColsToScan);
    const headerVals = headerRange.getValues() as (string | number | boolean)[][];

    outer:
    for (let r = 0; r < headerVals.length; r++) {
        for (let c = 0; c < headerVals[r].length; c++) {
            const cellVal = safeString(headerVals[r][c]);
            if (!cellVal) continue;

            // If the sheet uses labels like "Pickup Date: 10/09/2025", this will extract it.
            const extracted = extractISODateFromText(cellVal);
            if (extracted) {
                globalPickupDate = extracted;
                console.log("Detected global Pickup Date:", globalPickupDate);
                break outer;
            }

            // As a fallback, if a cell looks like just a date by itself:
            const iso = toISODate(cellVal);
            if (iso) {
                globalPickupDate = iso;
                console.log("Detected global Pickup Date (direct parse):", globalPickupD
ate);
                break outer;
            }

```

```

    }
  }
} catch (e) {
  console.log("Top-right date scan failed (non-fatal): " + e);
}

// Build header index map for target
const H: { [key: string]: number } = {};
for (let hi = 0; hi < TARGET_HEADERS.length; hi++) {
  H[TARGET_HEADERS[hi]] = hi;
}

// Try to find header row and detect column mapping
let headerRowIndex = -1;
let columnMapping: { [key: string]: number } = {};

const debugInfo: string[] = [];

for (let rowIdx = 0; rowIdx < Math.min(10, totalRows); rowIdx++) {
  const row = usedVals[rowIdx];
  const headers: string[] = [];

  for (let colIdx = 0; colIdx < totalCols; colIdx++) {
    headers.push(safeString(row[colIdx]));
  }

  // Check if this looks like a header row (contains common header terms)
  let headerCount = 0;
  for (let h = 0; h < headers.length; h++) {
    const header = headers[h].toLowerCase();
    if (
      header.includes("sender") || header.includes("receiver") || header.includes
("pickup") ||
      header.includes("delivery") || header.includes("pallet") || header.includes
("product")
    ) {
      headerCount++;
    }
  }
}

```

```

    }

    if (headerCount >= 3) { // Found a likely header row
        headerRowIndex = rowIdx;
        columnMapping = detectColumnMapping(headers);
        debugInfo.push("Found header row at: " + rowIdx);
        debugInfo.push("Column mapping: " + JSON.stringify(columnMapping));
        break;
    }
}

if (debugInfo.length > 0) {
    console.log(debugInfo.join("\n"));
}

// Process data rows - GROUP MULTI-LINE RECORDS TOGETHER
const dataStartRow = headerRowIndex !== -1 ? headerRowIndex + 1 : 0;
let rowsProcessed = 0;
const seen: Set<string> = new Set();

for (let dataRowIdx = dataStartRow; dataRowIdx < totalRows; dataRowIdx++) {
    const row = usedVals[dataRowIdx];

    // Quick bail-out for placeholder lines (no references present)
    if (looksLikeFillerInputRow(row)) {
        continue;
    }

    if (!isDataRow(row)) {
        continue;
    }

    // Prepare output row
    const outRow: string[] = new Array(TARGET_HEADERS.length).fill("");

    // IMPROVED MULTI-LINE SENDER DATA COLLECTION
    let senderData = "";

```

```

    if (columnMapping["senderName"] !== undefined) {
      // Collect all 3 lines of sender data
      for (let nextRow = 0; nextRow <= 2; nextRow++) {
        if (dataRowIdx + nextRow < totalRows) {
          const nextRowData = usedVals[dataRowIdx + nextRow];
          const addressLine = safeString(nextRowData[columnMapping["senderName"] || 0]);

          if (addressLine && isDataRow([addressLine])) {
            senderData += (senderData ? "\n" : "") + addressLine;
          }
        }
      }
    }

    // Parse the combined sender data
    const parsedSender = parseAddress(senderData);
    outRow[H["SenderName"]] = parsedSender.name;
    outRow[H["SenderDetails"]] = parsedSender.details;

    // Map other data based on detected columns
    if (columnMapping["pickupTimes"] !== undefined) {
      outRow[H["PickupTimes"]] = safeString(row[columnMapping["pickupTimes"]]);
    }

    // COLLECT MULTI-LINE RECEIVER DATA
    let receiverData = "";
    if (columnMapping["receiverName"] !== undefined) {
      // Collect all 3 lines of receiver data
      for (let nextRow = 0; nextRow <= 2; nextRow++) {
        if (dataRowIdx + nextRow < totalRows) {
          const nextRowData = usedVals[dataRowIdx + nextRow];
          const receiverLine = safeString(nextRowData[columnMapping["receiverName"] || 2]);

          if (receiverLine && isDataRow([receiverLine])) {
            receiverData += (receiverData ? "\n" : "") + receiverLine;
          }
        }
      }
    }

```

```

    }
  }
}

// Parse receiver address with improved logic
if (receiverData) {
  const parsedReceiver = parseAddress(receiverData);
  outRow[H["ReceiverName"]] = parsedReceiver.name;
  outRow[H["ReceiverSuburb"]] = parsedReceiver.details; // full details
  outRow[H["ReceiverState"]] = parsedReceiver.state;
}

// Map reference numbers
if (columnMapping["senderRef"] !== undefined) {
  outRow[H["SenderRefNumber"]] = safeString(row[columnMapping["senderRef"]]);
}
if (columnMapping["receiverRef"] !== undefined) {
  outRow[H["RefNumber"]] = safeString(row[columnMapping["receiverRef"]]);
}

// Map other fields with ISO DATE FORMATTING
if (columnMapping["productType"] !== undefined) {
  outRow[H["ProductType"]] = safeString(row[columnMapping["productType"]]);
}
if (columnMapping["palletType"] !== undefined) {
  outRow[H["PalletType"]] = safeString(row[columnMapping["palletType"]]);
}
if (columnMapping["transferType"] !== undefined) {
  outRow[H["TransferType"]] = safeString(row[columnMapping["transferType"]]);
}
if (columnMapping["palletQty"] !== undefined) {
  outRow[H["PalletQty"]] = safeString(row[columnMapping["palletQty"]]);
}
if (columnMapping["weight"] !== undefined) {
  outRow[H["Weight"]] = safeString(row[columnMapping["weight"]]);
}

```

```

    }
    if (columnMapping["palletDimensions"] !== undefined) {
        outRow[H["PalletDimensions"]] = safeString(row[columnMapping["palletDimensions"]]);
    }
    if (columnMapping["specialInstructions"] !== undefined) {
        outRow[H["SpecialInstructions"]] = safeString(row[columnMapping["specialInstructions"]]);
    }

    // DATE FIELDS - Ensure ISO format for Power Automate compatibility
    if (columnMapping["deliveryDate"] !== undefined) {
        outRow[H["ExpectedDeliveryDate"]] = toISODate(safeString(row[columnMapping["deliveryDate"]]));
    }

    // Extract pickup date from various row-level sources
    let pickupDate = "";
    if (columnMapping["pickupTimes"] !== undefined) {
        pickupDate = extractISODateFromText(safeString(row[columnMapping["pickupTimes"]]));
    }
    if (!pickupDate && columnMapping["senderRef"] !== undefined) {
        pickupDate = extractISODateFromText(safeString(row[columnMapping["senderRef"]]));
    }

    // Prefer row-level pickup date; otherwise use global (top-right) sheet pickup date
    if (pickupDate) {
        outRow[H["PickupDate"]] = pickupDate;
    } else if (globalPickupDate) {
        outRow[H["PickupDate"]] = globalPickupDate;
    }

    // Fallback: if no column mapping detected, try to use first few columns
    if (Object.keys(columnMapping).length === 0) {
        // Sender data (multi-line)

```

```

let senderDataFallback = "";
for (let nextRow = 0; nextRow <= 2; nextRow++) {
  if (dataRowIdx + nextRow < totalRows) {
    const nextRowData = usedVals[dataRowIdx + nextRow];
    const addressLine = safeString(nextRowData[0]);
    if (addressLine && isDataRow([addressLine])) {
      senderDataFallback += (senderDataFallback ? "\n" : "") + addressLin
e;
    }
  }
}

const parsedSenderFallback = parseAddress(senderDataFallback);
outRow[H["SenderName"]] = parsedSenderFallback.name;
outRow[H["SenderDetails"]] = parsedSenderFallback.details;

outRow[H["PickupTimes"]] = safeString(row[1]);

// Receiver data (multi-line)
let receiverDataFallback = "";
for (let nextRow = 0; nextRow <= 2; nextRow++) {
  if (dataRowIdx + nextRow < totalRows) {
    const nextRowData = usedVals[dataRowIdx + nextRow];
    const receiverLine = safeString(nextRowData[2]);
    if (receiverLine && isDataRow([receiverLine])) {
      receiverDataFallback += (receiverDataFallback ? "\n" : "") + receiverLi
ne;
    }
  }
}

const parsedReceiverFallback = parseAddress(receiverDataFallback);
outRow[H["ReceiverName"]] = parsedReceiverFallback.name;
outRow[H["ReceiverSuburb"]] = parsedReceiverFallback.details; // full details
outRow[H["ReceiverState"]] = parsedReceiverFallback.state;

if (totalCols > 5) outRow[H["ProductType"]] = safeString(row[5]);
if (totalCols > 6) outRow[H["PalletType"]] = safeString(row[6]);
if (totalCols > 7) outRow[H["TransferType"]] = safeString(row[7]);
if (totalCols > 8) outRow[H["PalletQty"]] = safeString(row[8]);

```

```

    if (totalCols > 10) outRow[H["PalletDimensions"]] = safeString(row[10]);
    if (totalCols > 11) outRow[H["Weight"]] = safeString(row[11]);
    if (totalCols > 12) {
      const senderRef = safeString(row[12]);
      outRow[H["SenderRefNumber"]] = senderRef;
      const dateFromRef = extractISODateFromText(senderRef);
      if (dateFromRef) outRow[H["PickupDate"]] = dateFromRef;
      else if (globalPickupDate) outRow[H["PickupDate"]] = globalPickupDate; //
← NEW fallback here as well
    } else {
      // If we still didn't put a PickupDate, try global
      if (!outRow[H["PickupDate"]] && globalPickupDate) {
        outRow[H["PickupDate"]] = globalPickupDate;
      }
    }
    if (totalCols > 13) outRow[H["RefNumber"]] = safeString(row[13]);
    if (totalCols > 14) outRow[H["ExpectedDeliveryDate"]] = toISODate(safeString
(row[14]));
  }

  // Check if this row has any data (ignoring placeholders)
  const hasData = outRow.some(cell => cell !== "" && !isPlaceholderCell(cell));

  // Stronger validity filter (drops the "PLEASE SELECT / AUTOFILL" repeats)
  if (hasData && isValidNormalizedRow(outRow, H)) {
    // De-duplicate strictly-similar repeats
    const dedupeKey = [
      outRow[H["SenderName"]],
      outRow[H["SenderDetails"]],
      outRow[H["PickupTimes"]],
      outRow[H["ReceiverName"]],
      outRow[H["ReceiverSuburb"]],
      outRow[H["ReceiverState"]],
      outRow[H["ProductType"]],
      outRow[H["PalletType"]],
      outRow[H["TransferType"]],
      outRow[H["PalletQty"]],
      outRow[H["Weight"]],

```



```

        outRow[H["PickupDate"]],
        outRow[H["ExpectedDeliveryDate"]]
    ].join("||");

    if (!seen.has(dedupeKey)) {
        dataRows.push(outRow);
        rowsProcessed++;
        seen.add(dedupeKey);

        // Skip the next 2 rows since we've already processed them as address line
s        dataRowIdx += 2;
    }
}
}

console.log(`Processed ${rowsProcessed} data rows`);

// Create normalized sheet
let normSheet = workbook.getWorksheet(NORMALIZED_SHEET);
if (!normSheet) {
    normSheet = workbook.addWorksheet(NORMALIZED_SHEET);
} else {
    const usedNorm = normSheet.getUsedRange();
    if (usedNorm) {
        usedNorm.clear(ExcelScript.ClearApplyTo.all);
    }
}

// Write header
const headerRange = normSheet.getRangeByIndexes(0, 0, 1, TARGET_HEADERS.l
length);
headerRange.setValues([TARGET_HEADERS]);

// Write data
if (dataRows.length > 0) {
    const dataRange = normSheet.getRangeByIndexes(1, 0, dataRows.length, TARG
ET_HEADERS.length);

```

```

    dataRange.setValues(dataRows);
}

// Create table
const existingTables = workbook.getTables();
for (let t = 0; t < existingTables.length; t++) {
    if (existingTables[t].getName() === TABLE_NAME) {
        existingTables[t].delete();
        break;
    }
}

if (dataRows.length > 0) {
    const lastRow = dataRows.length + 1;
    const lastCol = TARGET_HEADERS.length;
    const tableRange = "A1:" + colLetter(lastCol) + lastRow;
    const table = normSheet.addTable(tableRange, true);
    table.setName(TABLE_NAME);
}

const usedNormFinal = normSheet.getUsedRange();
if (usedNormFinal) {
    usedNormFinal.getFormat().autofitColumns();
}

return {
    ok: true,
    info: `Processed ${srcName}. Found ${rowsProcessed} data rows.`,
    rowsFound: rowsProcessed,
    tableName: TABLE_NAME,
    sheetName: normSheet.getName()
};
}

```